

复旦大学计算与智能创新学院 2025-2026 学年面向对象程序设计模拟试卷 01

课程名称：面向对象程序设计

考试形式：闭卷

考试时间：120 分钟

一、选择题（每题只选择一个答案；如选多个，不计分。每题 3 分，共 24 分）

1. 设有如下类定义：

```
#include <string>
class Node {
    const int id;
    std::string& ref;
public:
    // 在此处选择一个构造函数定义
};
```

若外部已有 `std::string s;`，则下列哪一个构造函数定义是正确的？

- A. `Node(int i, std::string& x) : id(i), ref(x) {}`
- B. `Node(int i, std::string& x) { id = i; ref = x; }`
- C. `Node(std::string& x, int i = 0) { id = i; }`
- D. `Node(int i, std::string x) : id(i), ref(x) {}`

2. 下列三组函数声明中，能够正确构成重载的是：

- (a) `int f(int);`
`int f(const int);`
- (b) `int g(int&);`
`int g(const int&);`
- (c) `int h(int*);`
`int h(const int*);`

- A. 只有 (a)
- B. (b) 和 (c)
- C. (a) 和 (c)
- D. (a)、(b)、(c)

3. 对下面程序，输出结果是：

```
#include <iostream>
using std::cout;
```

```

class Base {
public:
    virtual int f(int x = 1) const { return x + 1; }
    int g() const { return f(); }
};

class Derived : public Base {
public:
    int f(int x = 3) const { return x + 2; }
};

int main()
{
    Derived d;
    Base b = d;
    Base& r = d;
    cout << b.g() << " " << r.g() << " " << d.g();
    return 0;
}

```

- A. 2 2 2
- B. 2 3 3
- C. 4 4 4
- D. 2 4 4

4. 关于 vector、list 以及标准库算法，下列说法错误的是：

- A. vector 和 list 都可以保存一批同类型元素
- B. vector 和 list 都支持 begin()、end() 和 push_back()
- C. std::sort(v.begin(), v.end()) 可以用于 vector
- D. std::sort(lst.begin(), lst.end()) 可以直接用于 list

5. 关于 explicit 和类型转换，下列说法错误的是：

- A. 单参数构造函数若不加 explicit，可能参与隐式类型转换
- B. 若构造函数被声明为 explicit，则 A a = 3; 这种写法通常不能成立
- C. 若构造函数被声明为 explicit，则 A a(3); 这种直接初始化也不能成立
- D. 将 operator+ 写成非成员函数，常常有利于两侧操作数都参与自动类型转换

6. 一个类在构造函数中使用 new 申请动态内存。关于复制控制，下列说法正确的是：

- A. 只要定义了析构函数，就完全不需要定义复制构造函数和赋值运算符
- B. 编译器合成的复制构造函数和赋值运算符通常只会复制指针值，可能带来重复释放问题

- C. 只要程序里没有显式写出复制语句，就绝不会发生复制
- D. 对这种资源类来说，自我赋值检查是错误写法

7. 关于 shared_ptr、weak_ptr 和资源管理，下列说法错误的是：

- A. weak_ptr 不会增加对象的引用计数
- B. 可以通过 lock() 从 weak_ptr 获得一个 shared_ptr
- C. 当最后一个 shared_ptr 被销毁后，weak_ptr 会自动变成一个仍可安全解引用的普通指针
- D. weak_ptr 常被用来打破 shared_ptr 之间的循环引用

8. 关于抽象类与虚函数，下列说法正确的是：

- A. 只要类中有虚函数，这个类就一定是抽象类
- B. 抽象类不能定义构造函数
- C. 可以定义抽象类类型的指针或引用，用来指向派生类对象
- D. 纯虚函数在基类中绝对不能提供函数定义

二、程序阅读题（每题 5 分，共 20 分。说明：程序中省略了必要的头文件和 using 声明）

1. 写出下面程序的运行结果。

```
class Trace {
    int v;
public:
    Trace(int x = 0) : v(x) { cout << "A" << v << " "; }
    Trace(const Trace& rhs) : v(rhs.v + 1) { cout << "B" << v << " "; }
    ~Trace() { cout << "C" << v << " "; }
};

int main()
{
    Trace a(2);
    {
        Trace b = a;
        Trace c(5);
    }
    cout << "| ";
    return 0;
}
```

2. 写出下面程序的运行结果。

```
class Box {
    int v;
```

```

public:
    Box(int x = 0) : v(x) {}
    Box& operator+=(const Box& rhs) {
        v = v * 10 + rhs.v;
        return *this;
    }
    friend ostream& operator<<(ostream& os, const Box& b) {
        os << b.v;
        return os;
    }
};

```

```

template <class T>
T twice(T x)
{
    return x += x;
}

```

```

int main()
{
    Box b(3);
    cout << twice(b) << " " << b;
    return 0;
}

```

3. 写出下面程序的运行结果。

```

int main()
{
    vector<int> v;
    for (int i = 1; i <= 6; ++i)
        v.push_back(i);

    vector<int> w(2, 9);
    remove_copy_if(v.begin(), v.end(), back_inserter(w),
        [](int x) { return x % 2 == 0; });

    for (vector<int>::const_iterator it = w.begin(); it != w.end(); ++it)
        cout << *it << " ";
    return 0;
}

```

4. 写出下面程序的运行结果。

```
class Node {
    int id;
public:
    Node(int x) : id(x) { cout << "+" << id << " "; }
    ~Node() { cout << "-" << id << " "; }
};

int main()
{
    weak_ptr<Node> wp;
    {
        shared_ptr<Node> p(new Node(1));
        {
            shared_ptr<Node> q = p;
            cout << q.use_count() << " ";
            wp = q;
        }
        cout << p.use_count() << " ";
    }
    cout << (wp.expired() ? "E" : "N");
    return 0;
}
```

三、程序填空题（每空 3 分，共 24 分。说明：程序中省略了必要的头文件和 using 声明）

1. 下面的类在内部维护一段动态申请的整型数组。请完成复制构造函数、赋值运算符和析构函数，使其具有正确的深拷贝行为。

```
class Buffer {
public:
    Buffer(std::size_t n = 0) : sz(n), data(n ? new int[n] : 0) {}

    Buffer(const Buffer& rhs)
        : sz(rhs.sz), data( _____(1)_____ ) {
        _____(2)_____;
    }

    Buffer& operator=(const Buffer& rhs) {
        if (this != &rhs) {
            delete[] data;
            sz = rhs.sz;
        }
    }
}
```

```

        data = _____(3)_____;
        _____(4)_____;
    }
    return *this;
}

~Buffer() {
    _____(5)_____;
}

private:
    std::size_t sz;
    int* data;
};

```

2. 下面的代码定义了一个抽象基类和一个派生类，并通过基类接口完成多态复制。请填空。

```

class Shape {
public:
    virtual ~Shape() {}
    virtual double area() const = 0;
    virtual Shape* clone() const = 0;
};

class Circle : public Shape {
public:
    Circle(double r = 0) : radius(r) {}

    double area() const _____(6)_____ {
        return 3.14 * radius * radius;
    }

    Shape* clone() const {
        return _____(7)_____;
    }

private:
    double radius;
};

void add_shape(vector<Shape*>& shapes, const Shape& s)
{
    shapes.push_back( _____(8)_____ );
}

```

```
}
```

四、编程题 (共 32 分)

1. (12 分) 编写模板函数 `merge_unique(first1, last1, first2, last2, dest)`。

已知两个区间 `[first1, last1)` 和 `[first2, last2)` 中的元素都按非降序排列。请编写模板函数 `merge_unique`，将这两个区间合并到目标位置 `dest`，并满足以下要求：

1. 输出结果仍按非降序排列；
2. 若某个值在任意一个区间中出现了多次，或者同时出现在两个区间中，输出中都只保留一次；
3. 函数返回合并后写入结束位置对应的输出迭代器；
4. 不得调用标准库中的 `merge`、`unique` 或其它现成的合并算法；
5. 你的实现应当适用于 `vector`、`list` 等容器的迭代器。

以下为测试程序片段：

```
int main()
{
    vector<int> a = {1, 2, 2, 4, 7, 9};
    list<int> b = {2, 3, 4, 4, 8, 9};
    vector<int> c;

    merge_unique(a.begin(), a.end(), b.begin(), b.end(), back_inserter(c));

    for (vector<int>::const_iterator it = c.begin(); it != c.end(); ++it)
        cout << *it << " ";
    cout << endl;
    return 0;
}
```

该程序的输出为：

1 2 3 4 7 8 9

2. (20 分) 按要求实现抽象基类 `Shape`、派生类 `Rectangle` 和 `Circle`，以及管理多个图形对象的类 `Picture`。

要求如下：

1. 基类 `Shape` 含有图形名称 `name`，并声明以下成员：
 - 构造函数 `Shape(const string& n = "")`
 - 虚析构函数
 - 纯虚函数 `double area() const`
 - 纯虚函数 `void scale(double factor)`
 - 纯虚函数 `Shape* clone() const`
 - 虚函数 `void print(ostream& os) const`
2. 派生类 `Rectangle`：

- 数据成员: width、height
 - 构造函数: Rectangle(const string& n = "", double w = 0, double h = 0)
 - 面积为 width * height
 - scale(factor) 使长和宽都乘以 factor
3. 派生类 Circle:
- 数据成员: radius
 - 构造函数: Circle(const string& n = "", double r = 0)
 - 圆面积按 $3.14 * radius * radius$ 计算
 - scale(factor) 使半径乘以 factor
4. 类 Picture 用 vector<Shape*> 保存多个图形, 并支持:
- 默认构造函数
 - 复制构造函数, 要求深拷贝
 - 赋值运算符, 要求深拷贝并正确处理自我赋值
 - 析构函数, 负责释放所有动态对象
 - void add(const Shape& s): 将一个图形复制后加入容器
 - void scale_all(double factor): 把所有图形按同一比例缩放
 - double total_area() const: 返回所有图形面积之和
 - 友元输出运算符 operator<<, 按加入顺序逐行输出每个图形的名称和面积
5. 可以自行补充必要的辅助函数。

测试程序如下:

```
int main()
{
    Picture p;
    Rectangle r("RectA", 3, 4);
    Circle c("CircleA", 2);

    p.add(r);
    p.add(c);

    Picture q = p;
    q.scale_all(2);

    cout.setf(ios::fixed);
    cout.precision(2);

    cout << p.total_area() << endl;
    cout << q.total_area() << endl;
    cout << q;
    return 0;
}
```

程序的输出为:

24.56

98.24

RectA 48.00

CircleA 50.24